

Problem A. Italian Cuisine

Suppose the last cut piece is bounded by the directed segment from p_i to p_j , and the pineapple lies on the left side of the line $p_i p_j$.

Fix i . Among all valid j with $j > i$ (where indices are taken cyclically, so $p_{n+i} = p_i$), we should take the largest possible j . The predicate “the pineapple is on the left of the directed line $p_i p_j$ ” is monotone in the cyclic order, and as i increases, this largest valid j never decreases. Therefore all candidates can be maintained by a two-pointer scan.

The area computation only needs the sum of cross products over the boundary edges. The total time complexity is $\mathcal{O}(n)$.

Problem B. Generated String

First consider the easier case where every string is given explicitly. Build a trie of the strings. A string s is a prefix of a string t iff the vertex for t lies in the subtree of the vertex for s , which is an interval in DFS order. Building another trie for the reversed strings gives the same representation for suffix constraints. Thus, counting strings with a given prefix and suffix is a two-dimensional dominance query; after adding the time dimension, it becomes a three-dimensional dominance problem and can be solved in $\mathcal{O}(n \log^2 n)$ time.

For concatenations of substrings, the only missing part is how to build the trie. Sort all generated strings lexicographically, compute the LCP of adjacent strings, and build the compressed trie in the same way as a virtual tree. A naive comparison may cost $\mathcal{O}(k + l)$ for strings made of k and l pieces and may become quadratic. Two possible fixes are:

- use merge sort and compute adjacent LCP values during recursion, reusing them while merging;
- insert strings in increasing order of the number of pieces.

After preprocessing a suffix array so that LCPs of substrings are answered in $\mathcal{O}(1)$, these comparisons have total cost $\mathcal{O}(\sum k_i \log n)$. The total complexity is

$$\mathcal{O}\left(n \log n + q \log^2 q + \sum_i k_i \log q\right).$$

Problem C. Permutation

Use divide and conquer. Split the current range into two halves, determine for every value which half contains it, and recurse.

The interactor is non-adaptive: the hidden permutation is fixed in advance. Therefore we may randomize the order of the values. For two values x, y , ask a query whose first half is filled with x and whose second half is filled with y ; positions outside the current range can be filled arbitrarily with x or y . The answer has three cases:

- answer 0: x is in the second half and y is in the first half;
- answer 1: x and y are on the same side, but we do not know which side;
- answer 2: x is in the first half and y is in the second half.

Answers 0 and 2 classify both values. For answer 1, discard one of x, y and keep querying with the other.

Because of the initial shuffle, each value is approximately independently and uniformly distributed between the two halves. Each query classifies two values with probability $1/2$ and one value with probability $1/2$, so it classifies $3/2$ values in expectation. Hence one level uses about $2n/3$ queries, and the whole divide-and-conquer uses about $(2/3)n \log n$ queries. With pruning, such as stopping when one side is already full, the practical number of queries for $n = 1000$ is about 6000.

Problem D. Collect the Coins

Binary search the answer. For a fixed speed/limit, simulate the events in chronological order while maintaining the possible positions of the two robots.

At any time, the possible positions can always be represented as two intervals $[l_1, r_1]$ and $[l_2, r_2]$, without distinguishing which robot is which. Initially both intervals are $[1, 10^9]$. When time increases by t , both intervals expand by $v \cdot t$ in both directions. When a coin appears at position c :

- if neither interval contains c , the answer is impossible;
- if exactly one interval contains c , that interval becomes $[c, c]$;
- if both intervals contain c , either robot may collect the coin. The other robot's possible set is $[l_1, r_1] \cup [l_2, r_2]$, and since the two intervals intersect at c , this union is again an interval.

The feasibility check is linear, so the total complexity is $\mathcal{O}(n \log C)$.

Problem E. Normal Friends

Let the segment tree size be $n + 1$, and let the root split point be n . Then flipping the prefix $[1, x]$ is equivalent to finding the $(x + 1)$ -st leaf in DFS order and reversing all left subtrees on the path from that leaf to the root, from top to bottom, while also swapping left and right children inside those subtrees.

This operation can be implemented with a Link-Cut Tree style structure. First, we need to find the relevant chain. On every preferred path, maintain by a splay tree the left and right side subtrees in depth order, together with leaf counts and the number of nodes that have left or right children. To find the k -th DFS leaf on a chain, let L be the total number of leaves in all left side subtrees. If $k \leq L$, recurse into the appropriate left subtree; if $k = L + 1$, the leaf is the bottom of the chain; otherwise recurse into the appropriate right subtree with index $k - L - 1$.

After finding the leaf, perform an access operation. Since parent pointers are hard to maintain under future reversals, record during the search which preferred paths are crossed and which parent each path must be attached to. Processing from bottom to top, when attaching x to y , detach the current child z of y , split the left/right lists into two chains, insert z into the correct side list, remove x from the side list of y , concatenate the lists, and finally connect x to y .

The resulting preferred path gives the answer: it is the number of left subtrees on that path. Then perform the update by reversing the list of left children with a lazy reverse tag, and by mirroring every affected subtree. The mirroring is represented by swapping left and right child lists and lazily propagating this tag during later searches.

The access-count analysis of LCT still applies: each operation changes only $\mathcal{O}(\log n)$ light edges. The splay analysis is no longer exactly the classical LCT proof, but there is still an $\mathcal{O}(\log n)$ upper bound per relevant operation. The total complexity is at most $\mathcal{O}((n + q) \log^2 n)$, and the implementation is fast in practice.

Problem F. Triangle

Let x be the lexicographically largest of the three strings, and let the other two be y and z . Then $xy > z$ and $xz > y$ are automatic, so it is enough to make either $yz > x$ or $zy > x$ true. Assume $yz \geq zy$; this is equivalent to $y^\infty \geq z^\infty$, which adds one dominance constraint.

Now we need $yz > x$. Since $y \leq x$, the string y must be a prefix of x . Enumerate all pairs (x, y) where y is a prefix of x ; the total number of such pairs is bounded by the total input length. Let $y^{-1}x$ be the suffix remaining after removing this prefix. The possible z must satisfy

$$z^\infty \leq y^\infty, \quad x \geq z > y^{-1}x.$$

The second condition can be handled by sorting all string suffixes. For the first condition, sort strings by the order induced by infinite repetition. Directly comparing yz and zy may be quadratic overall. A correct comparison first checks whether one string is a prefix of the other; if not, compare directly, and otherwise use preprocessed LCP information from a Z-function or suffix array. Then one comparison costs $\mathcal{O}(\min(|y|, |z|))$ rather than $\mathcal{O}(|y| + |z|)$.

After the two order constraints are prepared, the remaining counting problem is two-dimensional dominance and can be solved in $\mathcal{O}(n \log n)$. Equal strings in the input must be grouped and counted carefully. The total complexity is

$$\mathcal{O}\left(\sum_i |S_i| \log n\right).$$

Problem G. Stop the Castle 2

Rephrase the task as placing $m - k$ obstacles so that as many attacking pairs of rooks as possible are separated.

Only adjacent rooks in the same row or the same column can attack each other, so there are only $\mathcal{O}(n)$ relevant pairs. If an empty cell can be placed between such a pair, one obstacle there blocks that attack. Let c be the number of blockable attacking pairs.

Some obstacles can block two attacking pairs at once: one horizontal pair and one vertical pair. We should maximize how often this happens. Build a bipartite graph whose vertices are attacking pairs. For every obstacle position that blocks one horizontal pair and one vertical pair, add an edge between the corresponding vertices. If the maximum matching size is x , then by placing $m - k$ obstacles we can block

$$\min\{c, m - k + \min(m - k, x)\}$$

attacking pairs. The maximum matching can be found by Dinic's algorithm or Hopcroft–Karp in $\mathcal{O}(m\sqrt{n})$ time.

Problem H. Intersection of Paths

For $k = 1$, the answer is the tree diameter. For general k , only edges whose two sides both contain at least k vertices matter; the desired value is the diameter obtained from these edges.

The queries are independent, so process them in decreasing order of k . As k decreases, edges are added. If there are no modifications, it is enough to maintain the diameter of the current connected component.

With edge-weight modifications, there are at least two approaches.

- Since almost all edge weights are unchanged, remove the modified edge, compute the diameters of the two components, and merge them. The diameter endpoints of the whole tree are among the endpoints of these two diameters. This requires supporting vertex additions and subtree diameters, which can be done by a segment tree over DFS order together with $\mathcal{O}(1)$ LCA queries.
- Use a dynamic-diameter segment tree directly. In an Euler tour, the diameter can be written as the maximum of

$$\text{dep}_u - 2\text{dep}_v + \text{dep}_w$$

over three, not necessarily distinct, vertices u, v, w in order. Changing an edge weight adds a value to a contiguous range of depths in the Euler order, so a segment tree with range add and the appropriate maximum expressions solves the update.

The total complexity is $\mathcal{O}((n + q) \log n)$.

Problem I. Find Yourself

First, every tree is determinable. Since a tree is bipartite, color it black and white and first make the monster's color the same as the root's color, possibly by spending one move. Maintain a vertex x such that the monster is somewhere in the subtree of x and has the same color as x . If x is a leaf, the game is finished. Otherwise, repeat until x has only one remaining child subtree: query x . If the monster is at x , stop. Otherwise the monster moves to the opposite color inside the subtree of x . Query one child subtree; if the monster is not in it, remove that subtree, and otherwise remove all other child subtrees. After the monster moves again, it is again somewhere in the remaining part with the same color as x . When only one child remains, query x and then descend to that child if necessary.

For graphs with cycles, note that if a graph H is not determinable, then any graph containing H as a subgraph is also not determinable. Experiments and the sample cases reveal several forbidden structures: cycles of length other than 4, certain dense or externally attached 4-cycles, and 4-cycles with adjacent length-2 chains attached.

The resulting structural characterization is as follows. Every biconnected component of a determinable graph must be a 4-cycle, or a spindle formed by several length-2 paths with the same two endpoints. We may simplify the graph further: if two degree-2 vertices have the same two neighbors, deleting one of them does not change the answer. After this reduction, every valid biconnected component is a 4-cycle, and these cycles must avoid the forbidden attachments described above.

The conditions are also sufficient. Root the block tree at a suitable vertex. In each 4-cycle, among the three non-root vertices, at most two have outside leaf attachments and at least one has no outside edge. A small case analysis shows how to query the root of the cycle and one adjacent vertex to reduce the monster's possible positions exactly as in the tree strategy.

All reductions and all checks can be implemented in $\mathcal{O}(n + m)$ time.

Problem J. Sheriruth

If a vertex has two outgoing edges to u and v , then u and v must be connected to each other. If either of them has outgoing edges, the whole set reachable from them becomes a clique. Repeatedly merging such cliques decomposes every connected component into one of the following forms:

- an in-arborescence;
- an in-functional graph with exactly one directed cycle;
- a clique, with several in-arborescences whose roots point to some vertices of the clique.

For an in-arborescence, each query reduces to an ancestor-descendant test. For an in-unicyclic component, we additionally need to know whether a vertex is on the cycle.

For a clique of size s , the number of possible internal paths is

$$\sum_{i=0}^{s-2} (s-2)^{\underline{i}},$$

where $a^{\underline{i}}$ denotes a falling factorial. When a path enters the clique from a tree root, one special case is whether the first clique vertex is already the target v ; in that case there is only one way. This value is computed once per clique, and the total clique size is at most n .

The total complexity is $\mathcal{O}((n + m + q) \log n)$; the logarithm comes from sorting and binary searches for edge existence, and can be removed with more careful implementation.

Problem K. Palindromic Polygon

Duplicate the cyclic list of points, obtaining a sequence of length $2n$. Then we need to choose a subsequence

$$i_1 < i_2 < \dots < i_k, \quad i_k - i_1 < n,$$

such that the corresponding color sequence $f_{i_1}, f_{i_2}, \dots, f_{i_k}$ is a palindrome, and the convex-hull area is maximized.

Use DP from the two ends. Let $F(l, r)$ be the best area when the current outer matched positions are l and r . A direct transition enumerates i, j with

$$l < i \leq j < r, \quad f_i = f_j,$$

and moves to $F(i, j)$, which is $\mathcal{O}(n^4)$.

The key observation is that if both intervals (l, i) and (j, r) contain a point with color f_i , then selecting such points cannot decrease the convex-hull area: it only adds two triangles. Therefore every useful transition has either i as the first occurrence of its color in the interval or j as the last occurrence. This reduces the useful transitions for each state to $\mathcal{O}(r - l)$ and gives an $\mathcal{O}(n^3)$ algorithm.

Problem L. Cosmic Travel

First consider a query with $l = r$. Build a binary trie of all a_i . Starting from the root, if the corresponding bit of l is 1, swap the two children. Then go to the left child if k is at most its size, otherwise subtract that size and go right.

For a general interval $[l, r]$, recursively split the interval by bits, in the same way as segment-tree decomposition. It remains to answer quickly when the current interval is a complete binary range. For each trie subtree and each $1 \leq k \leq$ its size, precompute for every $j \in [0, 2^d)$ the sum of the k -th smallest values of $a_i \oplus j$ inside that subtree, where d is the number of remaining bits. At each depth the number of stored states is $\mathcal{O}(n)$, so the total number of values is $\mathcal{O}(n \log C)$. Each value can be computed from the two children in $\mathcal{O}(1)$.

Thus the total complexity is $\mathcal{O}((n + q) \log C)$.

Problem M. The Quest for El Dorado

Run Dijkstra's algorithm with a lexicographic distance. A state distance is a pair (i, x) , meaning that the vertex can be reached no earlier than round i , and at least x distance has already been walked in that round. We minimize i first and x second.

For an edge (v, c, l) , if $a_i = c$ and $x + l \leq b_i$, we can traverse it in the same round and the new distance is $(i, x + l)$. Otherwise we must find the smallest $j > i$ such that

$$a_j = c, \quad b_j \geq l,$$

and then the new distance is (j, l) . This search can be supported by a segment tree or by binary searching over RMQ data.

The total complexity is $\mathcal{O}((n + m + k) \log(n + m + k))$.